

qstream

Wire Protocol Specification

Version 0.2 (Draft) — header, manifest request, piece request

Companion to `SPEC.md` (design & milestones). This document defines the on-the-wire format. Status: **Implemented** — M0, M1, M2.

1 Introduction

qstream is a peer-to-peer live video streaming protocol carried over plain UDP. One **master** node serves an HLS playlist (manifest + .ts segments); **peers** download segments and re-serve them to each other. Every node has a single UDP socket on which it both listens and sends.

UDP gives no ordering, no deduplication and no reliability — everything beyond “send a datagram” is defined here. This document specifies:

- the fixed *datagram header* (section 3),
- the *message catalog* (section 4),
- the *manifest request* happy path (section 5),
- the *piece (segment) request* happy path (section 6).

The flow-control and retransmission rules are described in `SPEC.md` §7; only the wire formats are fixed here.

2 Conventions

- All integers are **big-endian** (network byte order).
- Every datagram is exactly one message: fixed header + optional payload.
- Byte offsets are 0-based from the start of the datagram.
- Reserved or unused fields are sent as 0x00 and must be ignored on read.
- Example byte strings are written as space-separated hex, e.g. 51 53 54.

3 Datagram header

The header is fixed at **14 bytes** and present in every message.

0	1	2	3	4	5	6	7	8	9	10	11	12	13
magic	ver	type	flags	data length		transfer id		packet #		total			

Offset	Size	Field	Description
0	3	magic	“QST” (0x51 0x53 0x54). Rejects stray/garbage datagrams.
3	1	version	Protocol version, currently 0x02.
4	1	message type	One of the codes in section 4.
5	1	flags	ACK type for ACK messages (0x00 progress, 0x04 complete); else 0x00.
6	2	data length	Payload length in bytes (0..=65535). Datagram = 14 + this.
8	2	transfer id	Correlates all datagrams of one piece transfer; 0x0000 when unused.
10	2	packet number	1-based packet index within a transfer; 0x0000 when unused.
12	2	total packets	Total packets N of the transfer; 0x0000 when unused.

Validation. A receiver drops a datagram if the magic or version do not match, or if data length exceeds the remaining bytes of the datagram.

3.1 Field usage per message

Fields are only meaningful for certain message types; the others must be zero. A *transfer* is identified by its *transfer id* — chosen at random by the requester and echoed by the responder in every related datagram.

Message	flags	transfer id	packet #	total	payload
HANDSHAKE_REQUEST	—	—	—	—	node name (UTF-8)
HANDSHAKE_RESPONSE	—	—	—	—	node name (UTF-8)
MANIFEST_REQUEST	—	—	—	—	—
MANIFEST_RESPONSE	—	—	—	—	m3u8 contents
SEGMENT_REQUEST	—	✓ transfer	—	—	filename (UTF-8)
SEGMENT_CONTENTS	—	✓ transfer	✓ index	✓ N	file chunk ≤ 1400 B
SEGMENT_NOT_FOUND	—	✓ transfer	—	—	—
ACK	✓ type	✓ transfer	—	—	next range (start, count) — see §6

4 Message catalog

Code	Message	Direction	Status	Payload
0x01	HANDSHAKE_REQUEST	peer → master	done (M0)	node name (UTF-8)
0x02	HANDSHAKE_RESPONSE	master → peer	done (M0)	node name (UTF-8)
0x10	PING	any → any	planned	—
0x11	PONG	any → any	planned	—
0x20	MANIFEST_REQUEST	peer → master	done (M1)	—
0x21	MANIFEST_RESPONSE	master → peer	done (M1)	m3u8 contents
0x30	SEGMENT_REQUEST	any → any	spec'd (M2)	filename (UTF-8)
0x31	SEGMENT_CONTENTS	any → any	spec'd (M2)	file chunk ≤ 1400 B
0x32	SEGMENT_NOT_FOUND	any → any	spec'd (M2)	—
0x40	ACK	any → any	spec'd (M2)	next range (start, count)
0x50	PEERLIST_REQUEST	peer → master	planned	—
0x51	PEERLIST_RESPONSE	master → peer	planned	packed ip:port entries

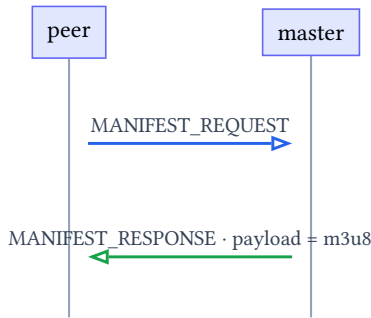
5 Manifest request (happy path)

The peer polls the master for the current HLS playlist. Polling is *stateless*: every request is answered with the latest manifest, there is no session.

5.1 Messages

Message	Format
MANIFEST_REQUEST	51 53 54 (magic) · 02 (version) · 20 (type) · 00 (flags) · 00 00 (data length) · 00 00 (transfer id) · 00 00 (packet #) · 00 00 (total) — 14 bytes, no payload.
MANIFEST_RESPONSE	Same header with type 21 and data length = manifest size; payload = raw m3u8 bytes. Example for the playlist #EXTM3U\n (8 bytes): 51 53 54 02 21 00 00 08 00 00 00 00 00 00 then 23 45 58 54 4D 33 55 0A.

5.2 Exchange



1. The peer sends `MANIFEST_REQUEST` (no payload, no transfer id).
2. The master re-reads its playlist file from disk and replies `MANIFEST_RESPONSE` with the raw contents.
3. The peer writes the response atomically (tmp + rename) to `<data-dir>/live.m3u8`.

Failure handling. If the master cannot read the manifest it replies with an empty payload (the peer keeps its previous copy). A timeout is retried on the next poll – the peer polls every 3 s by default.

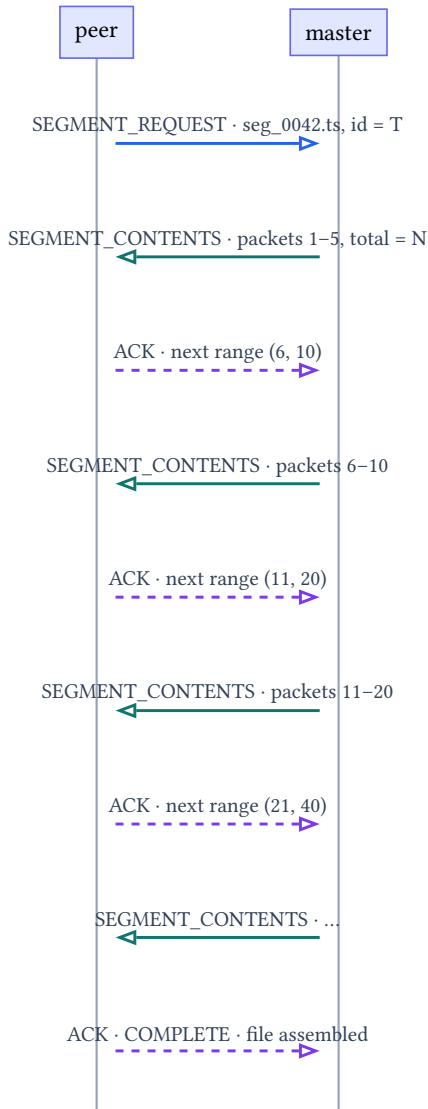
6 Piece (segment) request (happy path)

A piece is one `.ts` segment file, transferred as a sequence of packets. The transfer is **receiver-driven**: the peer explicitly requests the next packet range with every ACK, so there is no sender/receiver window synchronization to lose (see `SPEC.md §7`). `INITIAL_WINDOW = 5`, `MAX_WINDOW = 64`.

6.1 Messages

Message	Format
SEGMENT_REQUEST	Type 30 transfer id = fresh random value; payload = filename. Example requesting <code>seg_0042.ts</code> with transfer id <code>0x01A7</code> : 51 53 54 02 30 00 00 0B 01 A7 00 00 00 00 then 73 65 67 5F 30 30 34 32 2E 74 73.
SEGMENT_CONTENTS	Type 31 echoes transfer id; packet number = 1-based index; total packets = N payload = chunk of the file, ≤ 1400 bytes. First packet of a 60-packet file: 51 53 54 02 31 00 05 78 01 A7 00 01 00 3C followed by 1400 bytes of file data.
SEGMENT_NOT_FOUND	Type 32 echoes transfer id; no payload. The master does not have the requested file.
ACK	Type 40 echoes transfer id; payload = next range (start, count) as two big-endian u16 (4 bytes). start is the first packet wanted, count how many. Example: “send me packets 6..10” → 00 06 00 05. With flags = 0x04 and an empty payload the transfer is complete.

6.2 Exchange (happy path, window growth 5 → 10 → 20)



1. The peer sends `SEGMENT_REQUEST` for `seg_0042.ts` with a fresh *transfer id* `T`.
2. The master reads the file, computes $N = \max(1, \text{ceil}(\text{size}/1400))$ packets, and sends the first window: packets 1..5.
3. The peer learns `N` from the first content packet and replies `ACK (6, 10)`: the next window, doubled.
4. Windows double on every fully-received range ($5 \rightarrow 10 \rightarrow 20 \rightarrow 40 \rightarrow \dots$) up to `MAX_WINDOW = 64`.
5. As soon as the peer has all `N` packets it reassembles the file (out-of-order safe: packets are placed by their packet number), writes it atomically, and sends `ACK` with the **COMPLETE** flag.
6. The master frees the transfer state.

Loss handling (short version; full rules in `SPEC.md §7`): if the peer's current range does not fill within the quiet period, it re-sends the same `ACK` (retransmit request). If the master's ack timer fires it re-sends its last range — the peer deduplicates, so blind re-sends are safe. Both sides give up after 8 retries.

7 Appendix: transfer settings

Setting	Value	Meaning
<code>SEGMENT_PACKET_SIZE</code>	1400	bytes per content packet
<code>INITIAL_WINDOW</code>	5	first requested range size
<code>MAX_WINDOW</code>	64	largest range size
<code>PACE_INTERVAL_MS</code>	1	sleep between packets (rate limiter)
<code>WINDOW_QUIET_MS</code>	150	receiver quiet period before ACKing

FIRST_RESPONSE_TIMEOUT_MS	2000	give up if nothing arrives
WINDOW_RETRY_LIMIT	8	receiver re-request limit
ACK_RETRY_LIMIT	8	sender resend limit
MAX_CONCURRENT_TRANSFERS	16	per-node transfer registry bound